Project - High Level Design

on

Autonomous Energy Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Ansh Upadhyay	EN22CS301147
02	Anuroop Patidar	EN22CS301176
03	Arnika Jain	EN22CS301196
04	Arnoday Soni	EN22CS301197
05	Aryan Singh	EN22CS301214

Group Name: Group 01D2

Project Number: AAI – 13

Industry Mentor Name:

University Mentor Name: Prof. Ajeet Singh Rajput

Academic Year: 2026

Table Of Content

1. Introduction **3**
1.1 Scope of the Document 3
1.2 Intended Audience 3
1.3 System Overview 3
2. System Design **4**
2.1 Application Design 4
2.2 Process Flow 5
2.3 Information Flow 6
2.4 Components Design 7
2.5 Key Design Considerations 7
2.6 API Catalogue 8
3. Data Design **8**
3.1 Data Model 8
3.2 Data Access Mechanism 10
3.3 Data Retention Policies 10
3.4 Data Migration 10
4. Interfaces **10**
5. State and Session Management **10**
6. Caching **11**
7. Non-Functional Requirements **12**
7.1 Security Aspects 12
7.2 Performance Aspects 12
8. References **12**

1. Introduction

The High-Level Design (HLD) document provides an overview of the architecture and major components of the Autonomous Energy Researcher Agent system. It describes how the system is structured, how different modules interact with each other, and how data flows across the application.

The objective of this document is to present a clear and structured design of the system at a conceptual level before moving to detailed implementation. It focuses on system architecture, major components, data handling mechanisms, and non-functional requirements.

1.1 Scope of the Document

This document covers the high-level architecture of the Autonomous Energy Researcher Agent. It includes:

- Overall system architecture
- Major system modules and their interaction
- Application workflow
- Data design and storage mechanism
- System interfaces
- Security and performance aspects

The document does not include detailed code-level implementation, which is covered in the Low-Level Design (LLD).

1.2 Intended Audience

This document is intended for:

- Developers implementing the system
- Academic evaluators and mentors
- Students studying Agentic AI systems
- Researchers interested in autonomous AI agents

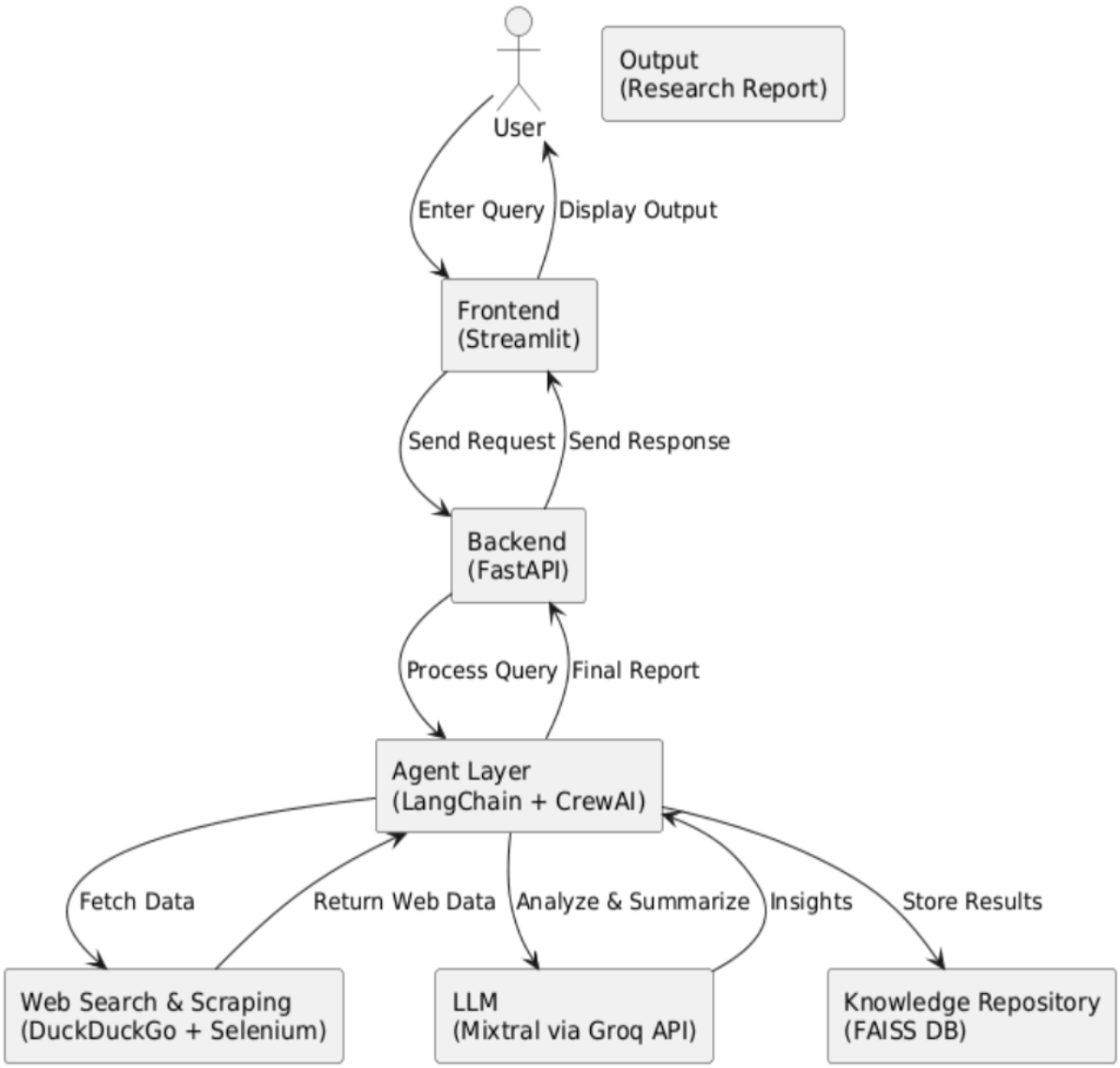
It helps the reader understand how the system is designed and how different modules work together.

1.3 System Overview

The Autonomous Energy Researcher Agent is an AI-powered multi-agent research system designed to automatically collect, analyze, and summarize energy-related information from the web.

The system uses multiple intelligent agents that work together to perform different tasks such as searching for data, analyzing information, and generating structured summaries. The final research results are stored in a knowledge repository for future use.

Figure 1: High-Level System Overview Diagram



2. System Design

The system follows a **modular multi-agent architecture** where different agents perform specific tasks. The system is designed to be scalable, flexible, and easy to maintain.

2.1 Application Design

The application is divided into the following high-level modules:

1. User Interface Layer

- Built using Streamlit
- Accepts user queries
- Displays research results

2. Backend Layer

- Implemented using FastAPI
- Handles communication between frontend and agents
- Manages API calls

3. Agent Layer

- Built using LangChain and CrewAI
- Contains three main agents:
    - Research Agent
    - Analysis Agent
    - Summary Agent

4. Data Collection Layer

- Uses DuckDuckGo Search
- Uses Selenium Web Scraper
- Extracts content from web pages

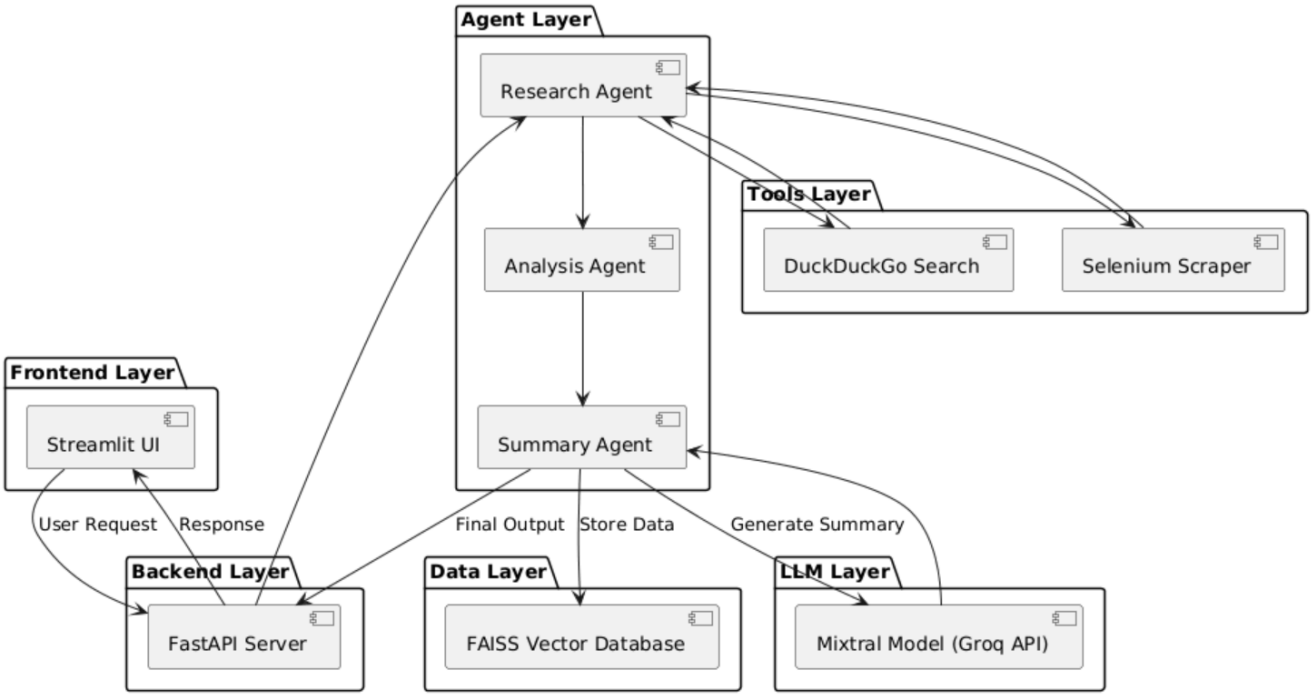
5. AI/LLM Layer

- Uses Mixtral-8x7B model via Groq API
- Responsible for reasoning and summarization

6. Knowledge Repository

- Uses FAISS Vector Database
- Stores research summaries
- Maintains historical research data

Figure 2: High-Level Architecture Diagram



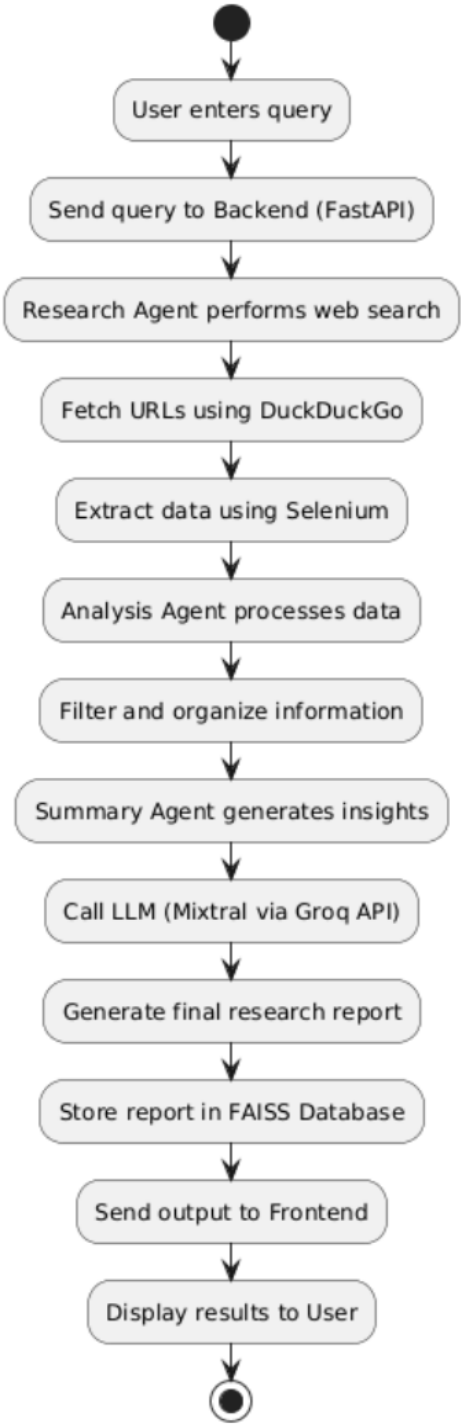
2.2 Process Flow

The system follows the below workflow:

1. User enters a query
2. Query is sent to the backend
3. Research Agent performs web search
4. Data is extracted from web sources
5. Analysis Agent processes and filters the data
6. Summary Agent generates structured insights
7. Final research report is generated

8. Report is stored in the knowledge repository
9. Output is displayed to the user

Figure 3: System Process Flow Diagram



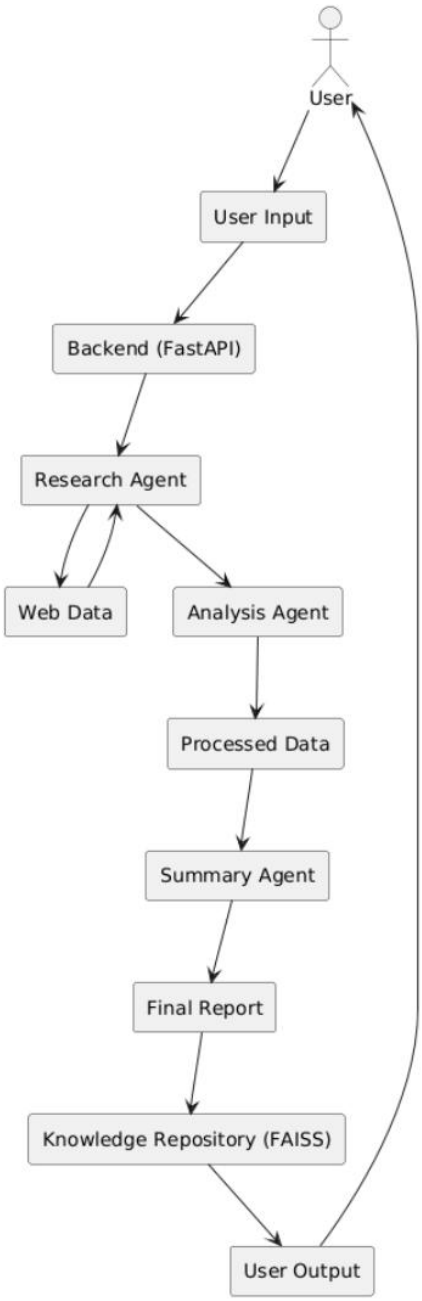
2.3 Information Flow

The information flows through the system in the following order:

User Input → Backend → Research Agent → Web Data → Analysis Agent → Processed Data → Summary Agent → Final Report → Knowledge Repository → User Output

---

Figure 4: Information Flow Diagram



2.4 Components Design

The system consists of the following main components:

Research Agent
Responsible for collecting relevant information from web sources.

Analysis Agent
Processes collected data and extracts meaningful insights.

Summary Agent
Generates structured summaries using the LLM.

Web Search Module
Uses DuckDuckGo to fetch relevant URLs.

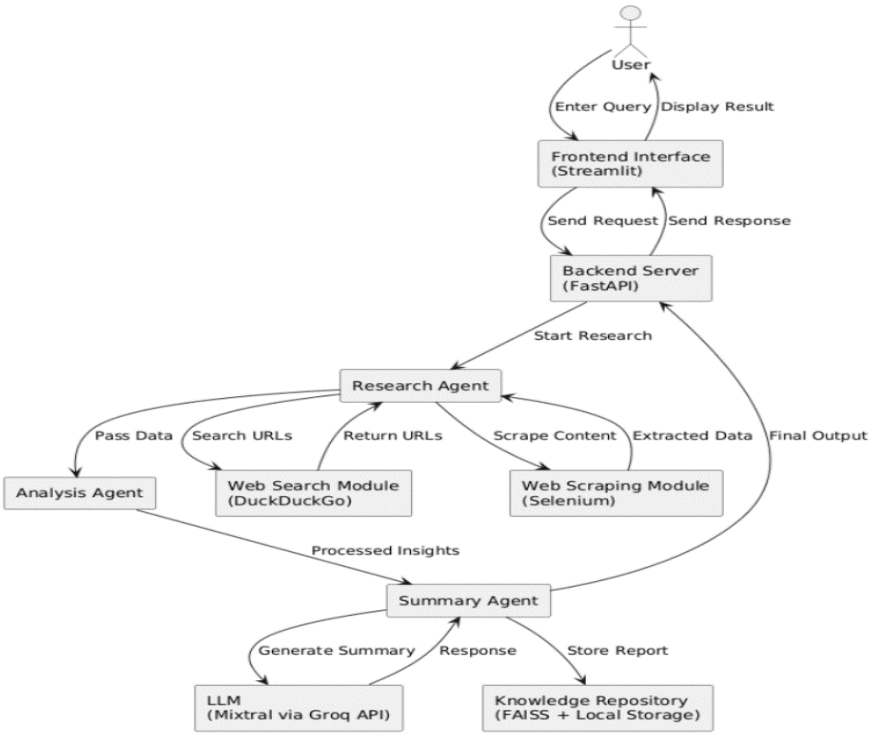
Web Scraping Module
Uses Selenium to extract content from web pages.

Knowledge Repository
Stores research reports using FAISS and local storage.

Backend Server
Handles communication between frontend and AI agents using FastAPI.

Frontend Interface
Built using Streamlit for user interaction.

Figure 5: Component Interaction Diagram



2.5 Key Design Considerations

Scalability

The system is designed using modular architecture, allowing new agents to be added easily.

Modularity

Each component (agents, tools, storage) works independently.

Flexibility

The system can be extended to other research domains beyond energy.

Performance

The system uses the Mixtral model via Groq API for fast response time.

Reliability

The system includes structured data flow and modular execution.

2.6 API Catalogue

1. LLM API (Groq API)

Input: Processed data / query
Output: Generated research summary

2. Web Search API (DuckDuckGo)

Input: User query
Output: List of relevant URLs

3. Web Scraping Module

Input: Web URL
Output: Extracted textual content

4. Knowledge Storage API

Input: Final research output
Output: Stored data in FAISS

---

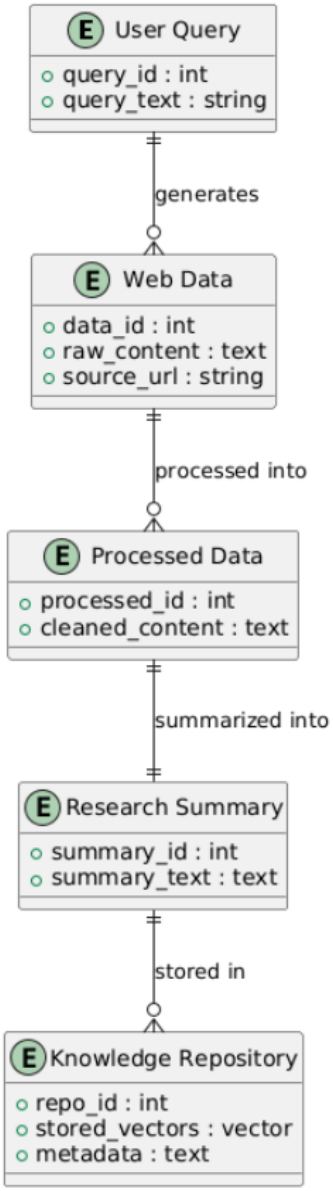
3. Data Design

3.1 Data Model

The system works with the following main data entities:

- User Query
- Web Data
- Processed Data
- Research Summary
- Knowledge Repository

Figure 6: High-Level Data Model Diagram



3.2 Data Access Mechanism

Data is accessed using:

- LangChain for embedding generation
- FAISS for similarity search
- Local storage for storing research reports

3.3 Data Retention Policies

The system stores research outputs for future use.
Old data can be updated or replaced based on new research results.

3.4 Data Migration

The system supports easy migration by storing research outputs in structured formats such as text or JSON files.

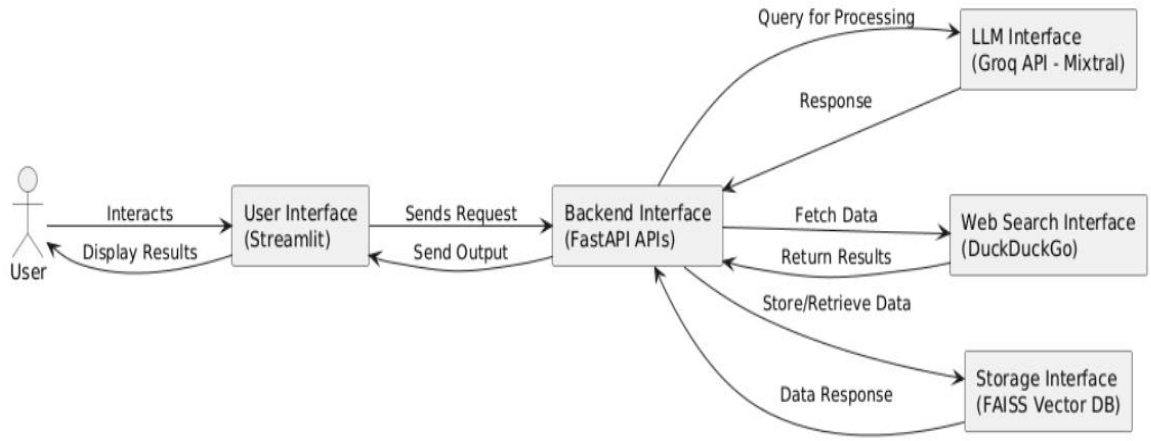
---

4. Interfaces

The system includes the following interfaces:

- User Interface (Streamlit)
- Backend Interface (FastAPI APIs)
- LLM Interface (Groq API)
- Web Search Interface (DuckDuckGo)
- Storage Interface (FAISS Vector Database)

Figure 7: System Interface Diagram



5. State and Session Management

The system maintains user sessions using Streamlit session state. Each query is processed independently, and research outputs are stored separately in the knowledge repository.

---

6. Caching

The system uses caching to improve performance:

- Cached search results
- Cached embeddings in FAISS
- Cached research summaries

This reduces repeated API calls and improves response time.

---

7. Non-Functional Requirements

7.1 Security Aspects

- No personal or sensitive data is stored
- API keys are stored securely using environment variables
- The system only processes publicly available web data

7.2 Performance Aspects

- Fast response using Mixtral via Groq API
- Efficient similarity search using FAISS
- Optimized data flow between agents

---

8. References

Below are the references written in proper academic format.

[1] LangChain Documentation. Available at: https://docs.langchain.com

[2] CrewAI Documentation. Available at: https://docs.crewai.com

[3] Groq API Documentation. Available at: https://console.groq.com/docs

[4] FAISS – Facebook AI Similarity Search Documentation. Available at: https://faiss.ai

[5] FastAPI Documentation. Available at: https://fastapi.tiangolo.com

[6] Streamlit Documentation. Available at: https://docs.streamlit.io

[7] DuckDuckGo Search API Documentation. Available at: https://duckduckgo.com

[8] Selenium WebDriver Documentation. Available at: https://www.selenium.dev/documentation

[9] Sentence Transformers Documentation. Available at: https://www.sbert.net